Командные оболочки, их роль и языки

Луцив Д. В.[1]

[1]СПбГУ



Ранний диалог: от перфокарт к интерактивности

- До середины 1960-х: пакетная обработка. Программа — колода перфокарт, результат — распечатка
- Прорыв: системы разделения времени (Time-Sharing)
  Много операторов, которые параллельно *что-то делают*
- CTSS (1961) и Teletype Model 33 позволили вести текстовый диалог с ЭВМ в реальном времени
- Ограничения: низвая скорость терминала (10 cps), отсутствие редактирования строки, эхо-печать

Эра текстового диалога и первые оболочки

1960-е

- Текстовый интерфейс (CLI) доминировал из-за дешевизны терминалов и ограниченности ресурсов для рендеринга графики
- Появление **Command-Line Interpreter**: прослойки между ядром ОС и пользователем
- Command-line interface взял на себя парсинг строки, поиск исполняемых файлов и управление потоками ввода-вывода
- Оболочка стала первым «языком программирования» для автоматизации рутины

Графический интерфейс и ООП: смена парадигмы

1970-е–80-е

- 1970-е: Xerox PARC и язык Smalltalk.
- Влияние на UX: переход от «команды к системе» к «манипуляции объектами» (метафора рабочего стола, WIMP: Windows, Icons, Menus, Pointer).
- Влияние ООП: инкапсуляция состояния. Пользователь взаимодействует с визуальными представителями объектов файловой системы
- Консоль ушла на второй план для массового пользователя, оставшись инструментом инженеров

Ренессанс консоли в мобильной и веб-разработке

2010-е

- Парадокс: чем выше уровень абстракции платформ (Web, iOS, Android), тем чаще разработчик возвращается в консоль
- Причины:
  1. Воспроизводимость и скриптуемость (Docker, npm, Webpack, Gradle)
  2. Удалённая разработка и CI/CD пайплайны не имеют GUI
  3. Мобильная отладка: ADB требует точного текстового контроля
- Консоль стала стандартом **профессионального** UX, обеспечивая максимальную пропускную способность команд
- Популярность легковесных языков разметки

Языковые модели и автоматизация: риски и вызовы

2020-е

- Современный этап: LLM оперируют объектами (файлами, API, БД) за пользователя по текстовому запросу
  - Markdown — любимый формат моделей
- Формат диалога сохраняется, но уровень абстракции повышается до «намерений» (intent-based computing)
- Устное общение (голосовое управление) удобнее для мобильных устройств, чем манипуляция объектами
- Появляется **Риск когнитивной деградации** (Cognitive Offloading):
  - человек снижает объём ручного труда (привинтить, подмести, смазать, покрасить)
  - человек перестаёт работать с объектами в прогаммах и на «рабочем столе» вручную, теряет навык навигации и точного синтаксиса
  - человек перестаёт ментально моделировать процесс, ограничиваясь формулировкой сиюминутных желаний, причём не точной (модели уже довольно понятливые)
  - для инженера это означает потерю способности к отладке и контура и структуры системы

Мейнфреймы и микрокомпьютеры

- JCL (OS/360/370): не интерактивная оболочка, а язык описания заданий (ресурсы, устройства) — декларативный предок скриптов
- Multics (1969): первая система, где оболочка была пользовательской программой, а не частью ядра
- CP/M (1974): `CCP` (Console Command Processor). Минималистичный, с базовыми командами (`DIR`, `ERA`)
- Эволюция в PC-мире: `COMMAND.COM` (MS-DOS) ⟶ `cmd.exe` (Windows). Пакетные файлы (`.bat`) с крайне ограниченными синтаксисом и возможностямиhttps://en.wikipedia.org/wiki/CP/Mhttps://en.wikipedia.org/wiki/CP/M

Интересные примеры: языки программирования как оболочки

- Sinclair BASIC и другие интерпретаторы BASIC для бытовых компьютеров содержали операторы взаимодействия с системой — `SAVE`, `LOAD`, `ERASE`, `LIST` (применительно к содержимому накопителя) и т.д., и использовались в том числе в роли командной оболочки
- Tcl: «Язык-оболочка» с легковесным синтаксисом, где всё есть строка, но с возможностями ближе к полноценным языкам и мощными возможностями расширения на C, нишевую популярность не потерял до сих пор
- **Python REPL**: часто используется как мощный калькулятор или оболочка для системных задач через `import os, sys`

sh
```
$ python3 -c "import os; print(os.listdir('.'))"
```

- **Node.js**: идеален для быстрой обработки JSON прямо в терминале

sh
```
$ node -e "console.log(JSON.parse(require('fs').readFileSync('data.json')).name)"
```

- **SQL-оболочки** (`psql`, `sqlite3`): интерактивная среда, где сам язык запросов является командной оболочкой, а мета-команды (начинающиеся с \) управляют окружением

Современные монстры: PowerShell, Elvish, Nushell

- PowerShell: революция в Windows. Потоки не текста, а *.NET-объектов*.

```sh
Get-Process | Where-Object { $_.CPU -gt 100 }
```

  несмотря на реализации доступные даже для Linux, популярность набирает медленно
- Elvish: современный POSIX-образный shell с поддержкой структур данных (списки, карты), и, как и в PowerShell, потоками таких объектов, продвинутым автодополнением; тоже экзотика
- Nushell: подход «данные как первая сущность». Вывод команд сразу парсится в структурированные таблицы, с которыми можно работать как в SQL

Философия Unix и унификация синтаксиса

- Unix Philosophy: «Пишите программы, которые делают что-то одно и делают это хорошо. Пишите программы, которые работают вместе».
- Унификация: в Unix **нет синтаксической разницы** между встроенной командой оболочки (builtin) и внешней программой.
- `cd` (builtin) и `ls` (external binary) вызываются одинаково: `command [args...]`.
- Код возврата (Exit status): любая команда возвращает целое число. `0` = успех, `1-255` = ошибка. Специальная переменная `$?` хранит код последней выполненной команды.
- sh

```sh
non-existent-cmd
echo $?
# Вывод: 127 (command not found)
```

Магия скобок: [– это команда test

- В Unix условие `if` проверяет не логическое выражение, а **код возврата** команды.
- Конструкция `[ "$a" = "$b" ]` — это не специальный синтаксис языка, а вызов внешней или встроенной команды с именем `[`!
- `test` / `[`: утилита, которая вычисляет выражение и возвращает 0 или 1.
- Символ `]` — это просто обязательный последний аргумент для команды `[`, чтобы скобки визуально закрывались.

sh

```sh
if [ -f "/etc/passwd" ]; then
    echo "File exists"
fi

# Абсолютно эквивалентная запись:
if test -f "/etc/passwd"; then
    echo "File exists"
fi
```

Перенаправление ввода-вывода и конвейеры (Pipes)

- Redirection: фундаментальный механизм взаимодействия процессов.
- > и >>: перенаправление стандартного вывода (stdout, fd 1) в файл.
- <: перенаправление стандартного ввода (stdin, fd 0) из файла.
- | (Pipe): соединение stdout одной программы с stdin другой. Данные текут как поток (stream), без промежуточных файлов.
- 2>&1: перенаправление стандартного потока ошибок (stderr, fd 2) туда же, куда направлен stdout. Критически важно для логирования.

sh

```sh
grep "ERROR" /var/log/syslog 2>&1 | sort | uniq > errors.txt
```

Переменные, кавычки и строгий режим

- Одна из главных опасностей Bash: word splitting и globbing при незакавыченных переменных.
- Всегда используйте двойные кавычки: `"$VAR"`.
- `set -euo pipefail` — «золотой стандарт» начала любого серьёзного скрипта:
  - `-e`: выход при ошибке любой команды.
  - `-u`: ошибка при использовании неопределённой переменной.
  - `-o pipefail`: ошибка, если любая команда в конвейере `|` завершилась с ошибкой.

sh

```sh
#!/bin/bash
set -euo pipefail

NAME="John Doe"
echo "$NAME"  # Правильно: выведет "John Doe"
echo $NAME    # Опасно: может разбиться на аргументы или вызвать подстановку имён файлов

# echo $UNDEFINED_VAR # Приведёт к немедленному завершению скрипта из-за -u
```

Циклы и подстановка команд (Command Substitution)

- Для вставки результата выполнения команды в строку используйте `$()`, а не устаревшие обратные кавычки .
- Цикл `for` в Bash итерируется по словам, разделённым пробелами (IFS).

sh

```sh
# Создание резервных копий всех .txt файлов
for file in $(ls *.txt); do
    echo "Processing: $file"
    cp "$file" "${file}.bak"
done

# Более безопасный вариант для имён с пробелами (используя find и while):
find . -name "*.txt" -print0 | while IFS= read -r -d '' file; do
    echo "Processing: $file"
done
```

Вопросы

- Как баланс между удобством GUI и мощностью CLI меняется в ваших текущих проектах?
- Сталкивались ли вы с проблемами «когнитивного оффлоадинга» при использовании AI-ассистентов для написания скриптов?
- Почему, на ваш взгляд, Nushell и Elvish пытаются изменить парадигму текстовых потоков на структурированные данные?